

FileGraphDB User Examples

FileGraphDB is a Python library that scans a folder of text files, builds relationships between files or chunks, and retrieves only the most relevant files for a question.

Install

```
# powershell
pip install filegraphdb
```

If testing from the local project folder before publishing:

```
# powershell
pip install -e .
```

Create A Demo Folder

```
# powershell
mkdir demo_docs

@"
The product launch was delayed because Acme Supplies missed the shipment.
Priya updated the beta release timeline.
"@ | Set-Content demo_docs\project_timeline.txt

@"
Acme Supplies confirmed the vendor shipment would arrive late.
The supplier delay affected the launch plan.
"@ | Set-Content demo_docs\vendor_delay.txt

@"
The budget review focused on cloud compute, model hosting, and GPU cost.
"@ | Set-Content demo_docs\budget_notes.txt
```

Example 1: Basic Retrieval

Purpose: build the graph and find files relevant to a question.

```
# python
from filegraphdb import FileGraphDB

graph = FileGraphDB("./demo_docs")
graph.build()

results = graph.retrieve("what caused the launch delay?", limit=3)

for result in results:
    print(result.document.rel_path, round(result.score, 3))
```

Example 2: Print LLM-Ready Context

Purpose: retrieve the best files and format them as context for an LLM.

```
# python
from filegraphdb import FileGraphDB

graph = FileGraphDB("./demo_docs")
graph.build()

context = graph.context_for_query(
    "what caused the launch delay?",
    limit=3,
    max_chars_per_file=1000,
)

print(context)
```

Example 3: Estimate Token Savings

Purpose: compare normal all-file prompting against FileGraphDB retrieval.

```
# python
from filegraphdb import FileGraphDB

graph = FileGraphDB("./demo_docs")
graph.build()

estimate = graph.estimate_token_savings(
    "what caused the launch delay?",
    limit=3,
    price_per_million_input_tokens=0.15,
)

print("All docs tokens:", estimate["all_docs_tokens"])
print("FileGraphDB tokens:", estimate["filegraph_tokens"])
print("Saved tokens:", estimate["saved_tokens"])
print("Saved percent:", estimate["saved_percent"])
print("Saved cost:", estimate.get("saved_cost"))
```

Example 4: Show Graph Relationships

Purpose: inspect the strongest file-to-file relationships.

```
# python
from filegraphdb import FileGraphDB

graph = FileGraphDB("./demo_docs")
graph.build()

for edge in graph.edges(limit=10):
    print(edge.source_path, "--", edge.type, "-->", edge.target_path)
    print("weight:", edge.weight)
    print("confidence:", edge.confidence)
    print("evidence:", edge.evidence)
    print()
```

Example 5: Related Files For One File

Purpose: ask what files are connected to a specific file.

```
# python
from filegraphdb import FileGraphDB

graph = FileGraphDB("./demo_docs")
```

```
graph.build()

edges = graph.related("project_timeline.txt", limit=5)

for edge in edges:
    print(edge.source_path, edge.type, edge.target_path, edge.weight)
```

Example 6: Custom Relationship Threshold

Purpose: control how many relationships are created.

```
# python
from filegraphdb import FileGraphDB

graph = FileGraphDB(
    "./demo_docs",
    min_relationship_score=0.20,
    max_edges_per_file=8,
)

summary = graph.build()

print(summary)
print("Edges:")

for edge in graph.edges(limit=20):
    print(edge.weight, edge.type, edge.source_path, edge.target_path)
```

Lower `min_relationship_score` creates more relationships but may add noise. Higher values create fewer but stronger relationships.

Example 7: Chunk Long Files

Purpose: split long documents into smaller chunks so the LLM does not need to read a whole large file.

```
# python
from filegraphdb import FileGraphDB

graph = FileGraphDB(
    "./demo_docs",
    chunk_word_threshold=2000,
    chunk_words=800,
    chunk_overlap=120,
)

summary = graph.build()
print(summary)

results = graph.retrieve("supplier delay beta release", limit=5)

for result in results:
    print(result.document.rel_path, round(result.score, 3))
```

Long files may appear as:

```
# text
big_report.txt
big_report.txt#chunk-0001
big_report.txt#chunk-0002
```

Internally the graph keeps both levels:

```
# text
File node: big_report.txt
Chunk node: big_report.txt#chunk-0001
Edge: big_report.txt --CONTAINS--> big_report.txt#chunk-0001
```

Example 8: Disable Chunking

Purpose: treat every file as one document, even if it is long.

```
# python
from filegraphdb import FileGraphDB

graph = FileGraphDB(
    "./demo_docs",
    chunk_word_threshold=10**12,
)

graph.build()

for result in graph.retrieve("supplier delay beta release", limit=5):
    print(result.document.rel_path, round(result.score, 3))
```

Example 9: Use An Open-Source Embedding Model

Purpose: use stronger semantic embeddings instead of the default lightweight LSA vectors.

Install optional model dependencies:

```
# powershell
pip install "filegraphdb[models]"
```

Python:

```
# python
from filegraphdb import FileGraphDB

graph = FileGraphDB(
    "./demo_docs",
    use_sentence_transformers=True,
    embedding_model="sentence-transformers/all-MiniLM-L6-v2",
)

graph.build()

results = graph.retrieve("what caused the launch delay?", limit=3)

for result in results:
    print(result.document.rel_path, round(result.score, 3), result.reason)
```

Example 10: Build Once, Query Many Times

Purpose: pay the graph-building cost once, then run many searches.

```
# python
from filegraphdb import FileGraphDB
```

```

graph = FileGraphDB("./demo_docs")
graph.build()

queries = [
    "what caused the launch delay?",
    "who updated the timeline?",
    "what documents mention cloud compute?",
]

for query in queries:
    print("\nQUERY:", query)

    for result in graph.retrieve(query, limit=3):
        print(" ", result.document.rel_path, round(result.score, 3))

```

Example 11: Send Context To Your Own LLM

Purpose: use FileGraphDB as a retrieval layer before an LLM call.

```

# python
from filegraphdb import FileGraphDB

graph = FileGraphDB("./demo_docs")
graph.build()

question = "what caused the launch delay?"
context = graph.context_for_query(question, limit=3)

prompt = f"""
Answer using only this context.

Question:
{question}

Context:
{context}
"""

print(prompt)

```

Example 12: Minimal Function Wrapper

Purpose: wrap FileGraphDB in a reusable helper function.

```

# python
from filegraphdb import FileGraphDB

def search_folder(folder: str, question: str, limit: int = 3):
    graph = FileGraphDB(folder)
    graph.build()
    return graph.retrieve(question, limit=limit)

results = search_folder("./demo_docs", "what caused the launch delay?")

for result in results:
    print(result.document.rel_path, result.score)

```

Example 13: Visualize The Graph

Purpose: create an interactive HTML bubble graph from the installed library.

First build the graph:

```
# powershell
filegraph --folder demo_docs build
```

Then create the visualization:

```
# powershell
filegraph --folder demo_docs visualize --out demo_graph.html --limit-edges 300
```

Open `demo_graph.html` in a browser.

The visualization shows:

- bubbles as files or chunks
- lines as relationships
- bubble size by connection count
- line thickness by relationship strength
- relationship type filters
- search by filename
- click node to see keywords, topics, and entities
- click edge to see weight, confidence, and evidence

CLI Quick Commands

```
# powershell
filegraph --folder demo_docs build
filegraph --folder demo_docs edges
filegraph --folder demo_docs search "what caused the launch delay?"
filegraph --folder demo_docs context "what caused the launch delay?"
filegraph --folder demo_docs estimate "what caused the launch delay?" --limit 3
filegraph --folder demo_docs visualize --out demo_graph.html
```

What Users Should Understand

FileGraphDB does not force the LLM to read every file. It builds a relationship graph first, retrieves the most relevant files or chunks, and then those selected files can be sent to an LLM.

The main benefit is:

```
# text
build once, query many times
fewer tokens
faster answers
more focused context
```